CSE231: Operating Systems – Assignment 3

Topic: *SimpleScheduler – A Process Scheduler in C from Scratch*

Group Members:

- **Garv Jain (2024216)**
- **Aryan Kalpesh Pancholi (2024122)**

Overview

The document conveys or describes the contribution made by each member of the group in the Assignment (*SimpleScheduler – A Process Scheduler in C from Scratch using the round robin scheduling approach).*

Garv Jain

Major Contributions :

- Implemented the SimpleShell for taking user inputs and also handled the submit ./a.out command.
- Handled errors and implemented code for command parsing .
- Implemented the ready queue display and the output formatting along with the process summary on termination.
- Contributed to writing the **documentation.**
- Assisted with **basic debugging and verification**, running multiple simple test cases to confirm functionality.

Aryan Kalpesh Pancholi

Major Contributions :

- Implemented the **scheduler daemon.**

- Implemented the **inter-process signaling mechanism** .
- Implemented the **process control logic**
- Implemented synchronization between SimpleShell and SimpleScheduler.

Joint Contributions

- Decided on the communication strategy between the shell and scheduler.

- Collaboratively ensured **error handling**, **process cleanup**, and **termination conditions**.

- Collaboratively tested multiple executables

Implementation of Simple Scheduler

The **SimpleScheduler** is a basic CPU process scheduler implemented in C to simulate how an operating system manages multiple jobs. It is designed to enqueue, execute, and manage processes in a controlled and time-sliced manner.

1. Structure

The scheduler maintains a **queue of jobs**, which contains essential information such as :

- Process ID (PID)
- Command or executable name
- Arrival time and start time
- Burst time or runtime
- Remaining time
- Completion status

2. Scheduling Logic

We have used the Round- Robin approach which states that every job gets a fixed time slice for execution.

- When a job finishes within its slice, it is marked as **Completed.**
- If not finished, it is **re-queued** to the end of the queue to wait for its next turn.

3. Process Execution

Each job is executing the fork() and exec() calls.

- The parent process handles job management, queue updates, and timing.
- The child process runs the actual executable.

4. Time Management

The scheduler calculates and prints timing metrics for each job:

- **Arrival Time**

- **Start Time**
- **End Time**
- **Completion Time**
- **Waiting Time**

5. Error Handling

- Invalid job files or commands
- Queue overflows (beyond max jobs)
- Child process termination fails.

Github Repo Link - Repo_Link